Multihop LoRa OTA Firmware Updates for APRS Networks

Experimental results, architecture, and further development
Prepared for Jon N7UV — updated 30 October 2026

1. Objective

Update ESP32 LoRa APRS iGates and digipeaters over RF, without an IP connection or physical intervention at the remote target. The longer-term goal is to distribute an image through several relays to one or more devices, with recovery after interruption and restoration of normal APRS operation.

An initial implementation of controlled distribution has now passed laboratory testing: one controller, one relay, and one target, using block groups, local repair, and resumption after a target restart. Multiple paths and distribution to several targets remain to be developed.

2. Test results

The tests used TTGO LoRa32 V2.1 boards with ESP32 processors and SX1278 radios. The relayed setup is **F4MLV-2 → F4MLV-10 → F4MLV-15**. The prototype is integrated into the iGate firmware containing RXT, native-frame support, and standalone PPM decoding.

The common fast profile is **BW500 kHz, SF7, CR4/5, 2 dBm**, giving **21.875 kbps nominal**. The initial bench profile is 434.100 MHz, BW125/SF12/CR4/5; the fast profile uses 434.500 MHz. These settings are specific to the laboratory setup.

Test	Image, bytes	Transfer and verification	Useful throughput
Direct link, target Wi-Fi off	1,744,480	26 min 38 s	1,092 bytes/s
One relay, ACK per block	1,760,528	1 h 08 min 18 s	430 bytes/s
One relay, groups and persistent resume	1,764,720	53 min 24 s	551 bytes/s

Durations cover firmware data transfer and image verification, not initial setup, initial profile negotiation, activation, or boot confirmation. The final test includes **73.24 seconds of deliberate recovery after a restart during the transfer**.

In the relayed setup, grouped transfer reduced the observed duration by about **22%** and increased useful throughput by about **28%**. The images are similar but not identical in size. This compares two successive tests, not a guaranteed improvement for every network or a direct measurement of cumulative airtime.

The target had both Wi-Fi and its access point disabled. Authenticated RF replies checked this state before and after the update. Relay Wi-Fi remained enabled for management. Firmware blocks and target commands travelled over RF. Initial provisioning and independent flash readback used USB.

3. What was verified

The complete image was written to the inactive ESP32 partition, checked using SHA-256, and validated as an ESP32 application image. An independent USB readback of that partition matches the transmitted file byte for byte.

Activation remains a separate authenticated command. Two returns to the previous firmware were observed: after an injected reset before confirmation, and after activation with confirmation withheld beyond the deadline. The new firmware was then activated and confirmed over RF. After another restart, it still answered through the relay with Wi-Fi off.

Switching to the fast profile and returning to the initial profile worked. Checks before the transfer also showed no target response to a direct controller command or while the relay was paused; replies resumed when the relay was enabled.

APRS text, native, and PPM reception had been checked in earlier direct-link tests. Those reception checks were not repeated in the final grouped-transfer test.

4. How grouped transfer works

The controller sends a group of **32 blocks of 128 bytes**, forming a **4,096-byte flash page**, without waiting for an ACK after each block. The final page may be shorter.

The relay listens to the group and caches the signed frames in RAM. On the controller's request, it forwards them to the target and asks for a reception summary containing a 32-bit bitmap. The controller and relay therefore take turns transmitting.

If the target is missing blocks held in the relay's cache, the relay repairs them locally. Remaining holes are reported to the controller, which sends only the blocks still missing. The cache holds **one group**, not an entire image, and is lost when the relay restarts.

Over the complete transfer, the relay counted **94 local block repairs** and the controller **17 requested block retransmissions**. One loss had been deliberately introduced on each link. These counters are neither an RF loss rate nor a diagnosis of the cause of losses.

5. Persistent resume: what survives

The target accepts out-of-order blocks within the current group and checks duplicates. It writes a complete page to the inactive partition. Only then does an NVS checkpoint record the image identity and durable offset. Blocks from an incomplete page remain in RAM.

The test committed **8,192 bytes**, then received **2,048 additional bytes** without completing the next page. An RF diagnostic command reset the target. After the nodes returned to their initial profiles and a fresh authenticated session was established, the target recovered the **8,192 committed bytes**. Only the incomplete page was retransmitted; committed pages were not sent again.

Session expiry preserves this progress; explicit cancellation clears it. Resumption uses fresh authentication rather than reusing the old nonce. The client also implements recovery after link loss, but physically restarting the relay during a transfer still needs testing.

Target reset was tested on hardware. Power cuts during page programming or checkpoint writes were only simulated in the C++ test bench; hardware power-loss testing remains necessary.

6. The two project architectures

Architecture 1 — Controlled distribution

A controller selects participants and schedules block delivery. Relays can cache blocks; receivers report holes for selective repair.

The prototype now implements an initial version: **one fixed path, one relay, one target, and a one-group cache**. Shared distribution to several targets, larger caches, and chains of several intermediate relays have not yet been implemented.

Architecture 2 — Multiple paths to a target

Two or more paths could maintain delivery when a link becomes unavailable. This requires coordinated transmissions, duplicate handling, and separate path state.

This extension remains to be developed. It supplements controlled distribution rather than replacing it. The recommendation remains to use Architecture 1 as the foundation and add alternate paths where their benefit is demonstrated.

7. Control and profile switching

Preparation, profile changes, activation, and confirmation use authenticated binary OTA commands. “Normal APRS profile” means the stored radio settings, not commands carried as ordinary APRS messages. The bench temporarily uses its own initial profile.

The controller prepares the session on that initial profile. The target acknowledges switching; the relay forwards the acknowledgment, then each node switches to the fast profile after its local delay. The controller checks the link before sending data. After image verification, participants return to the initial profile for activation and confirmation.

A node with one radio cannot keep listening to the initial profile while using the fast profile. Return commands during transfer therefore use the fast profile. A local inactivity lease restores stored settings if the link disappears; recovery does not depend on receiving a final command. A restart also reloads stored radio settings.

This mechanism works for the tested route. Coordinating several relays or different profiles per link is a separate extension.

8. Authentication, integrity, and boot

The protocol already includes controller and target identities, a persistent arming counter, a session nonce, block offsets and lengths, an authenticated manifest, and HMACs. The relay envelope adds the transmitting hop’s identity and separate transport authentication; the relay preserves the target’s end-to-end authenticator unchanged.

The manifest identifies image size, SHA-256, and the expected laboratory board. SHA-256 checks the content; manifest authentication protects the expected digest. The ESP32 image is checked before activation, and selecting the new firmware remains separate from receiving it.

RF boot confirmation and application rollback are independent of download resumption. An incomplete image must never be selected for boot. Extending this to other boards and partition layouts requires their own compatibility checks.

9. Hardware and data rates

BW500/SF7/CR4/5 can provide a common profile for SX1278 and SX1262 devices. The tests described here used SX1278 radios only.

SX1262 supports SF5: BW500/SF5/CR4/5 gives **62.5 kbps nominal**. SX1278 does not support SF5, so that rate has not been tested. Different profiles per link could optimize a mixed network, but they are not required to start with a compatible common profile.

These physical rates are not useful firmware throughput: headers, two hops, transmit/receive changes, reception summaries, and repairs all contribute to transfer duration.

10. Initial installation at remote sites

The OTA path inspected in Ricardo's firmware uses ElegantOTA and the ESP32 Web server. It carries firmware over IP and does not receive this protocol's blocks over LoRa.

An existing node must therefore first receive compatible firmware through Web OTA where reachable or local flashing. The experimental prototype described here has not been published as a deployment release for isolated sites.

11. Remaining validation

Topic	Status
One relay, complete image, target Wi-Fi off	Validated in the laboratory
Groups, cache, and selective repair	Validated over the full transfer
Resume after target reset	Validated on hardware
Power cuts during flash/NVS writes	Simulated on host; hardware tests pending
Relay restart or outage during transfer	Still to test
Several successive relays	Still to implement and test
Alternate paths and multiple targets	Still to implement and test
SX1262/SF5 and different profiles per link	Not measured
Remote sites, hidden nodes, asymmetry, busy channel	Field tests pending

Power-loss recovery, path handling, and coexistence with APRS traffic need testing before treating this as ready for inaccessible sites.

12. Current position

The first working form of Architecture 1 has been demonstrated on a two-hop path. Grouped transfer, local repair, and persistent resume work together over a complete image, with verified integrity, separate activation, RF confirmation, and rollback.

The next work concerns real write interruptions and relay failures, followed by extension to several relays, paths, and targets. The overall project architecture has not yet been fully validated.

The experimental code and the 30 October test report are available on branch `experiment/lora-ota`. The tested code is `8241529`, and the report is `043af4d`. This publishes experimental sources and results, not a firmware release offered for deployment.